

Advanced Machine Learning

Lecture 4: Convolutional Neural Networks (CNN)

Jens-Peter M. Zemke
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

5 November 2024



Outline

Repetition

- Starting values
- Variations on SGD
- Regularization

Convolutional Neural Networks (CNN)

- Motivation
- Convolutions
- Pooling
- Feedforward & backpropagation
- Keras/TensorFlow example

Outline

Repetition

- Starting values
- Variations on SGD
- Regularization

Convolutional Neural Networks (CNN)

- Motivation
- Convolutions
- Pooling
- Feedforward & backpropagation
- Keras/TensorFlow example

Three techniques for choosing starting weights

Zero start: very simple

- ▶ too much structure preserved, can not generalize
- ▶ all neurons in a hidden layer behave like one single neuron

Known values: best approach

- ▶ first layers often are universal, only later layers are trained new

Random start: generic

- ▶ generalizes
- ▶ for deep FNN careful selection of normalization depending on activation necessary

Optimizers

SGD: exists as plain SGD, batch GD, minibatch SGD

Momentum: decaying average of gradients as update

NAG: like momentum, but with predictor-corrector step

Adagrad: adaptive weighting of learning rates

RMSprop: decaying average of squared gradients as weighting

Adam: combination of momentum and RMSprop, decaying averages of first (gradients) and second (squared gradients) moments for update and weighting, respectively

Several variations exist: Adadelata, Nadam, Adamax, AMSGrad, AdamNC, AdamW

Preventing overfitting

Comes in three flavours:

weight decay: L^2 and L^1 regularization, corresponds to classical Tikhonov regularization

minibatch normalization: learns mean and variance of probability distribution and scales minibatch data linearly to exhibit this behaviour

randomization: drop neurons (Dropout) or connections (DropConnect)

Outline

Repetition

- Starting values
- Variations on SGD
- Regularization

Convolutional Neural Networks (CNN)

- Motivation
- Convolutions
- Pooling
- Feedforward & backpropagation
- Keras/TensorFlow example

Data we can not treat in a satisfactory manner

Certain signals, namely

- ▶ audio signals (1D),
- ▶ images (2D),
- ▶ medical data, e.g., voxels (3D),
- ▶ voxels changing in time (4D), change of airflow around an airplane (4D),
- ▶ Cauchy stress tensor (9D), ...

Data we can not treat in a satisfactory manner

Certain signals, namely

- ▶ audio signals (1D),
- ▶ images (2D),
- ▶ medical data, e.g., voxels (3D),
- ▶ voxels changing in time (4D), change of airflow around an airplane (4D),
- ▶ Cauchy stress tensor (9D), ...

share common properties:

- ▶ stored as multi-dimensional arrays (common: 1D to 4D, in general d D for large d possible)
- ▶ order matters, e.g., sounds in succession, neighbouring pixels, dark spots in the lung, ...
- ▶ several channels, e.g., stereo signal: left & right; image: RGB, CMYK; ...
- ▶ typically high dimensional if to be of any use

Problems and the way towards a solution

Problems:

- ▶ **spatial distribution** is **neglected**, any reordering would give an equivalent FNN. Contradicts human experience
- ▶ **relations between channels** are **neglected**
- ▶ **fully connected FNN** would have an **immense amount of weights** to train, not feasible

Problems and the way towards a solution

Problems:

- ▶ **spatial distribution** is **neglected**, any reordering would give an equivalent FNN. Contradicts human experience
- ▶ **relations between channels** are **neglected**
- ▶ **fully connected FNN** would have an **immense amount of weights** to train, not feasible

Ideas for a solution:

- ▶ we need **weights that are mostly zero** (sparseness),
- ▶ that **take neighbouring information into account**,
- ▶ and **do not depend** too much **on the precise absolute location**, more on the relative locations of several data points.

Problems and the way towards a solution

Problems:

- ▶ **spatial distribution** is **neglected**, any reordering would give an equivalent FNN. Contradicts human experience
- ▶ **relations between channels** are **neglected**
- ▶ **fully connected FNN** would have an **immense amount of weights** to train, not feasible

Ideas for a solution:

- ▶ we need **weights that are mostly zero** (sparseness),
- ▶ that **take neighbouring information into account**,
- ▶ and **do not depend** too much **on the precise absolute location**, more on the relative locations of several data points.

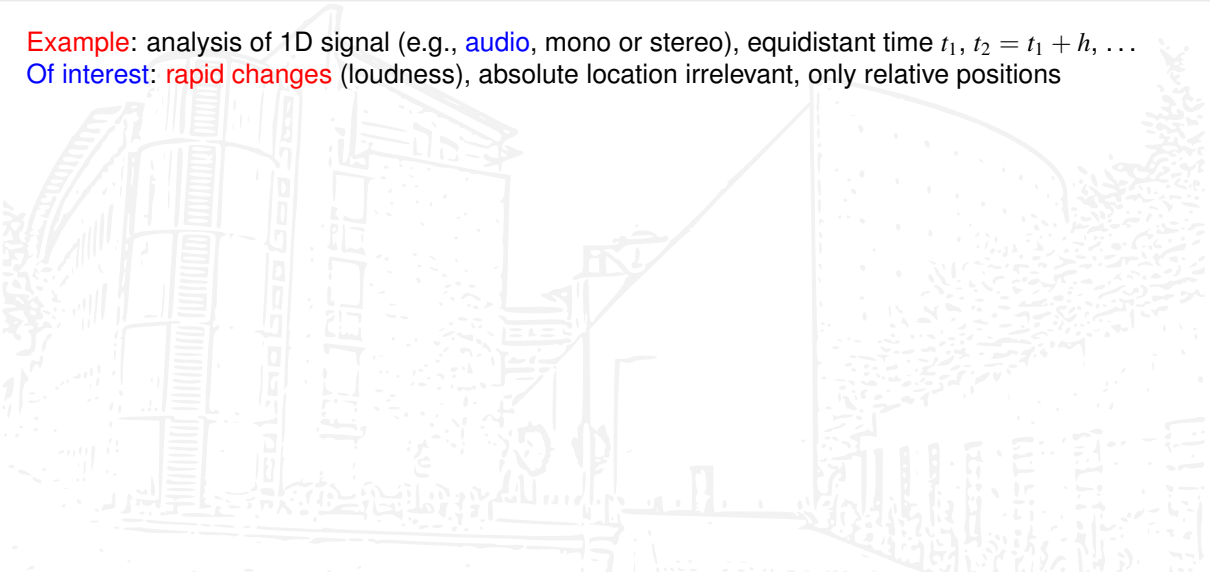
How? $\rightsquigarrow$ Let's take a look at **signal and image processing**

“Mathematische Bildverarbeitung”, Marko Lindner

1D signals: audio, light intensity, ...

Example: analysis of 1D signal (e.g., **audio**, mono or stereo), equidistant time $t_1, t_2 = t_1 + h, \dots$

Of interest: **rapid changes** (loudness), absolute location irrelevant, only relative positions

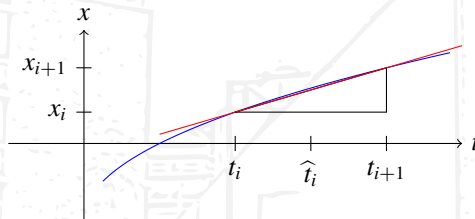


1D signals: audio, light intensity, ...

Example: analysis of 1D signal (e.g., **audio**, mono or stereo), equidistant time $t_1, t_2 = t_1 + h, \dots$

Of interest: **rapid changes** (loudness), absolute location irrelevant, only relative positions

Idea: use **approximate derivative**:

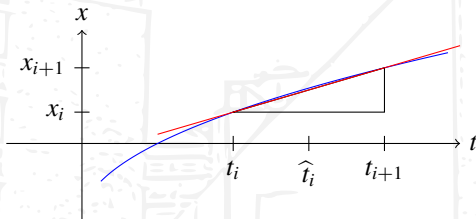


1D signals: audio, light intensity, ...

Example: analysis of 1D signal (e.g., **audio**, mono or stereo), equidistant time $t_1, t_2 = t_1 + h, \dots$

Of interest: **rapid changes** (loudness), absolute location irrelevant, only relative positions

Idea: use **approximate derivative**:



Numerical: **first order difference approximation**

$$x'(\hat{t}_i) \approx \frac{x_{i+1} - x_i}{t_{i+1} - t_i} = \frac{x_{i+1} - x_i}{h} = \sum_{k=0}^1 x_{i+k} d_k, \quad \mathbf{d} := \begin{pmatrix} d_0 \\ d_1 \end{pmatrix} = \begin{pmatrix} -1 \\ 1 \end{pmatrix} / h.$$

1D cross-correlation vs. 1D convolution

Given by so-called **cross-correlation** $\star$

$$x'(\hat{t}_i) \approx \frac{x_{i+1} - x_i}{h} = \sum_{k=0}^1 x_{i+k} d_k =: (\mathbf{x} \star \mathbf{d})_i$$

1D cross-correlation vs. 1D convolution

Given by so-called **cross-correlation** $\star$ or **convolution** $*$,

$$x'(\hat{t}_i) \approx \frac{x_{i+1} - x_i}{h} = \sum_{k=0}^1 x_{i+k} d_k =: (\mathbf{x} \star \mathbf{d})_i = \sum_{k=0}^1 x_{i+1-k} s_k =: (\mathbf{x} * \mathbf{s})_{i+1}, \quad \mathbf{s} := \begin{pmatrix} 1 \\ -1 \end{pmatrix} / h.$$

1D cross-correlation vs. 1D convolution

Given by so-called **cross-correlation** $\star$ or **convolution** $*$,

$$x'(\hat{t}_i) \approx \frac{x_{i+1} - x_i}{h} = \sum_{k=0}^1 x_{i+k} d_k =: (\mathbf{x} \star \mathbf{d})_i = \sum_{k=0}^1 x_{i+1-k} s_k =: (\mathbf{x} * \mathbf{s})_{i+1}, \quad \mathbf{s} := \begin{pmatrix} 1 \\ -1 \end{pmatrix} / h.$$

The vectors $\mathbf{s}$ and $\mathbf{d}$ are called **filters** or **kernels** and are related by **flipping the entries**,

$$\mathbf{s} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} / h, \quad \mathbf{d} = \begin{pmatrix} -1 \\ 1 \end{pmatrix} / h = \mathbf{F}_2 \mathbf{s} := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \mathbf{s}.$$

1D cross-correlation vs. 1D convolution

Given by so-called **cross-correlation** $\star$ or **convolution** $*$,

$$x'(\hat{t}_i) \approx \frac{x_{i+1} - x_i}{h} = \sum_{k=0}^1 x_{i+k} d_k =: (\mathbf{x} \star \mathbf{d})_i = \sum_{k=0}^1 x_{i+1-k} s_k =: (\mathbf{x} * \mathbf{s})_{i+1}, \quad \mathbf{s} := \begin{pmatrix} 1 \\ -1 \end{pmatrix} / h.$$

The vectors $\mathbf{s}$ and $\mathbf{d}$ are called **filters** or **kernels** and are related by **flipping the entries**,

$$\mathbf{s} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} / h, \quad \mathbf{d} = \begin{pmatrix} -1 \\ 1 \end{pmatrix} / h = \mathbf{F}_2 \mathbf{s} := \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \mathbf{s}.$$

In **stochastics** **cross-correlation** gives the **difference** of the probabilities $\mathbf{p}$ and $\mathbf{q}$ of two independent integer valued random variables X and Y ,

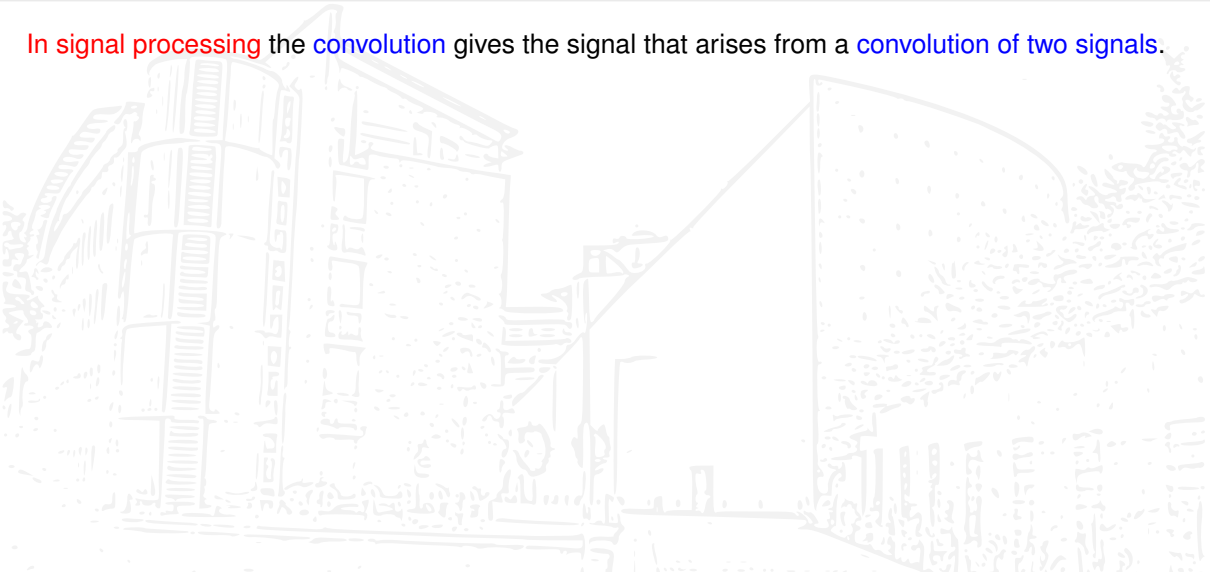
$$(\mathbf{p} \star \mathbf{q})_i = \sum_k p_{i+k} q_k = P[X - Y = i],$$

convolution gives the **sum**

$$(\mathbf{p} * \mathbf{q})_i = \sum_k p_{i-k} q_k = P[X + Y = i].$$

Convolution of polynomials/series, Cauchy product

In **signal processing** the **convolution** gives the signal that arises from a **convolution of two signals**.



Convolution of polynomials/series, Cauchy product

In **signal processing** the **convolution** gives the signal that arises from a **convolution of two signals**.

Prototype: **multiplication of polynomials** (series, where most coefficients are zero) and series,

$$\sum_{i=0}^{\infty} c_i x^i := \left(\sum_{n=0}^{\infty} a_n x^n \right) \left(\sum_{n=0}^{\infty} b_n x^n \right) = \sum_{i=0}^{\infty} \left(\sum_{k=0}^i a_{i-k} b_k \right) x^i, \quad \Rightarrow \quad \mathbf{c} = \mathbf{a} * \mathbf{b}.$$

The coefficients of the product are given by the **Cauchy product**, which is a convolution of the coefficients.

Convolution of polynomials/series, Cauchy product

In **signal processing** the **convolution** gives the signal that arises from a **convolution of two signals**.

Prototype: **multiplication of polynomials** (series, where most coefficients are zero) and series,

$$\sum_{i=0}^{\infty} c_i x^i := \left(\sum_{n=0}^{\infty} a_n x^n \right) \left(\sum_{n=0}^{\infty} b_n x^n \right) = \sum_{i=0}^{\infty} \left(\sum_{k=0}^i a_{i-k} b_k \right) x^i, \quad \Rightarrow \quad \mathbf{c} = \mathbf{a} * \mathbf{b}.$$

The coefficients of the product are given by the **Cauchy product**, which is a convolution of the coefficients.

There are **other natural kernels/filters**:

- ▶ computing **curvature** by $\mathbf{d} = \mathbf{s} = [-1, 2, -1]/h^2$, e.g., 1D **Laplacian**
- ▶ **averaging** noisy signals by (cheap) **low-pass filter** $\mathbf{d} = \mathbf{s} = [1, 1, 1]/3, [1, 1, \dots, 1]/n$

The last two mentioned filters are **symmetric** (cross-correlation = convolution).

Convolution as linear operation; structured weights

All approximate derivatives at midpoints:

$$\mathbf{y} := \begin{pmatrix} x'(\hat{t}_1) \\ \vdots \\ x'(\hat{t}_{n-1}) \end{pmatrix} = \frac{1}{h} \begin{pmatrix} -1 & 1 & & \\ & \ddots & \ddots & \\ & & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} =: \mathbf{T}\mathbf{x}, \quad \text{weight } \mathbf{W} = \mathbf{T} \text{ sparse Toeplitz}$$

Convolution/cross-correlation is a linear operation $\rightsquigarrow$ structured sparse weight matrix

Convolution as linear operation; structured weights

All approximate derivatives at midpoints:

$$\mathbf{y} := \begin{pmatrix} x'(\hat{t}_1) \\ \vdots \\ x'(\hat{t}_{n-1}) \end{pmatrix} = \frac{1}{h} \begin{pmatrix} -1 & 1 & & \\ & \ddots & \ddots & \\ & & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} =: \mathbf{T}\mathbf{x}, \quad \text{weight } \mathbf{W} = \mathbf{T} \text{ sparse Toeplitz}$$

Convolution/cross-correlation is a linear operation $\rightsquigarrow$ structured sparse weight matrix

We only computed the part where the kernel completely overlapped the data. Convolution is defined also for partial overlap, where we pad the input by zeros.

Convolution as linear operation; structured weights

All approximate derivatives at midpoints:

$$\mathbf{y} := \begin{pmatrix} x'(\hat{t}_1) \\ \vdots \\ x'(\hat{t}_{n-1}) \end{pmatrix} = \frac{1}{h} \begin{pmatrix} -1 & 1 & & \\ & \ddots & \ddots & \\ & & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} =: \mathbf{T}\mathbf{x}, \quad \text{weight } \mathbf{W} = \mathbf{T} \text{ sparse Toeplitz}$$

Convolution/cross-correlation is a linear operation $\rightsquigarrow$ structured sparse weight matrix

We only computed the part where the kernel completely overlapped the data. Convolution is defined also for partial overlap, where we pad the input by zeros.

If input $\mathbf{x} \in \mathbb{R}^n$ and kernel $\mathbf{f} \in \mathbb{R}^k$ then partial output $\mathbf{y}_p \in \mathbb{R}^{n-k+1}$ and full output $\mathbf{y}_f \in \mathbb{R}^{n+k-1}$. We remark that roles of input and kernel are interchangeable, here kernel low-dimensional $k \ll n$.

Convolution as linear operation; structured weights

In general, for input $\mathbf{x} \in \mathbb{R}^n$ and kernel $\mathbf{f} \in \mathbb{R}^k$, we define

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_k \\ \vdots \\ y_n \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \mathbf{x} * \mathbf{f} = \mathbf{T}\mathbf{x} := \begin{pmatrix} f_1 & & & & \\ \vdots & \ddots & & & \\ f_k & \cdots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \cdots & f_1 \\ & & & \ddots & \vdots \\ & & & & f_k \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \quad \mathbf{T} \in \mathbb{R}^{(n+k-1) \times n}.$$

Convolution as linear operation; structured weights

In general, for input $\mathbf{x} \in \mathbb{R}^n$ and kernel $\mathbf{f} \in \mathbb{R}^k$, we define

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_k \\ \vdots \\ y_n \\ \vdots \\ y_{n+k-1} \end{pmatrix} = \mathbf{x} * \mathbf{f} = \mathbf{T}\mathbf{x} := \begin{pmatrix} f_1 & & & & \\ \vdots & \ddots & & & \\ f_k & \dots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \dots & f_1 \\ & & & \ddots & \vdots \\ & & & & f_k \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \\ x_n \end{pmatrix}, \quad \mathbf{T} \in \mathbb{R}^{(n+k-1) \times n}.$$

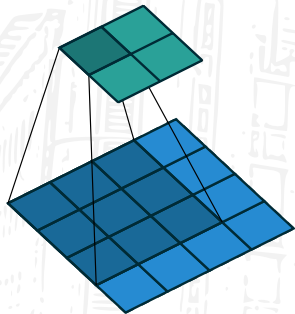
Python:

- ▶ `np.convolve(x, f, mode='full')` (default)
- ▶ `np.convolve(x, f, mode='valid')` only the middle part

2D convolutions

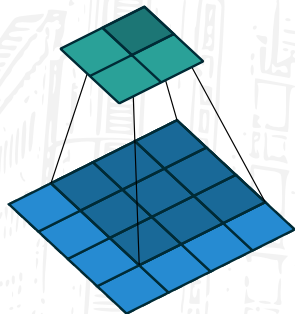
In 2D, input and output are **matrices**, here:

- ▶ input $\mathbf{X} \in \mathbb{R}^{4 \times 4}$ (light blue), $i = 4$
- ▶ output $\mathbf{Y} \in \mathbb{R}^{2 \times 2}$ (light green), $o = 2$



Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions



In 2D, input and output are **matrices**, here:

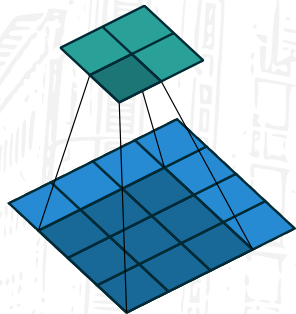
- ▶ input $\mathbf{X} \in \mathbb{R}^{4 \times 4}$ (light blue), $i = 4$
- ▶ output $\mathbf{Y} \in \mathbb{R}^{2 \times 2}$ (light green), $o = 2$

Also **kernel/filter is a matrix**, here

- ▶ $\mathbf{F} \in \mathbb{R}^{3 \times 3}$ (shadow on input), $k = 3$, $o = i - k + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions



In 2D, input and output are **matrices**, here:

- ▶ input $\mathbf{X} \in \mathbb{R}^{4 \times 4}$ (light blue), $i = 4$
- ▶ output $\mathbf{Y} \in \mathbb{R}^{2 \times 2}$ (light green), $o = 2$

Also **kernel/filter is a matrix**, here

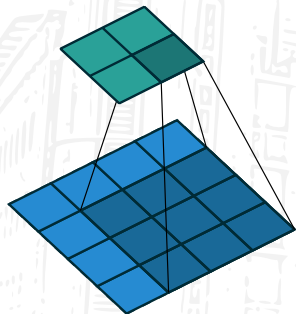
- ▶ $\mathbf{F} \in \mathbb{R}^{3 \times 3}$ (shadow on input), $k = 3$, $o = i - k + 1$

Cross-correlation:

- ▶ overlap, multiply, sum

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions



In 2D, input and output are **matrices**, here:

- ▶ input $\mathbf{X} \in \mathbb{R}^{4 \times 4}$ (light blue), $i = 4$
- ▶ output $\mathbf{Y} \in \mathbb{R}^{2 \times 2}$ (light green), $o = 2$

Also **kernel/filter is a matrix**, here

- ▶ $\mathbf{F} \in \mathbb{R}^{3 \times 3}$ (shadow on input), $k = 3$, $o = i - k + 1$

Cross-correlation:

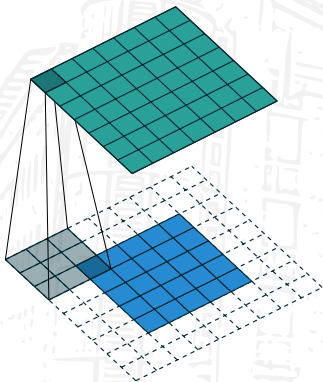
- ▶ overlap, multiply, sum

Convolution:

- ▶ flip kernel horizontally & vertically
- ▶ i.e., rotate by 180 degrees

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

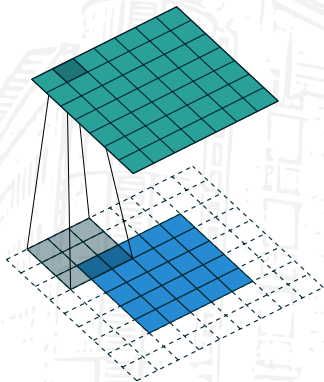
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

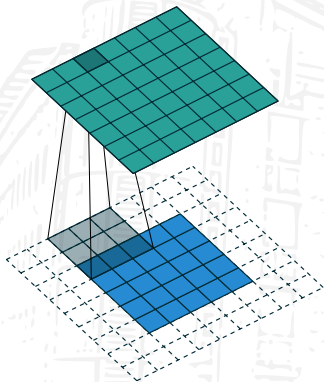
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

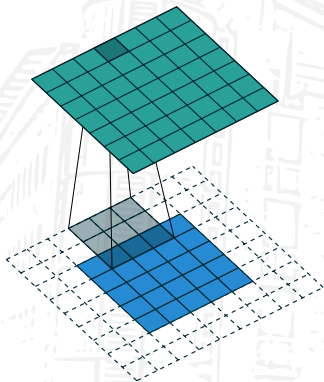
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

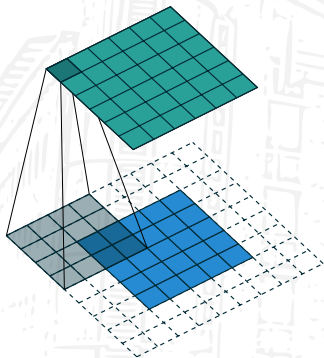
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

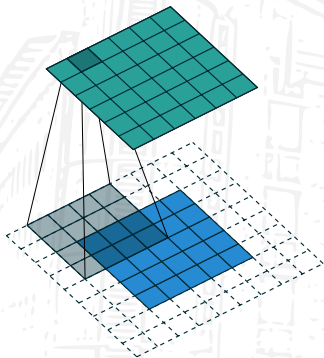
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

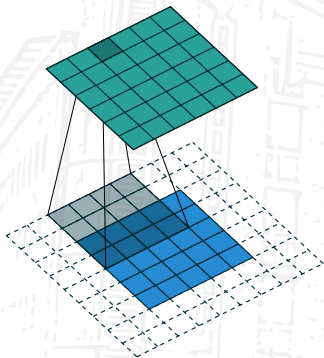
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

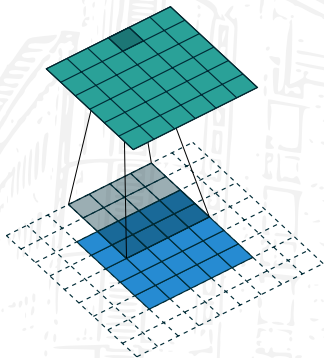
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

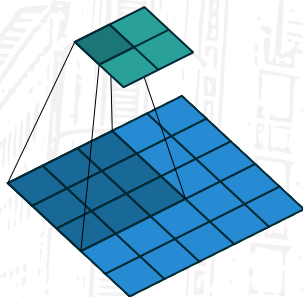
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

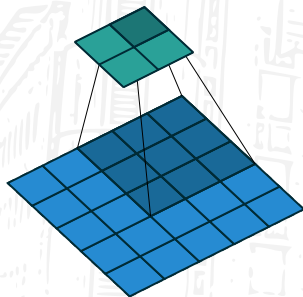
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

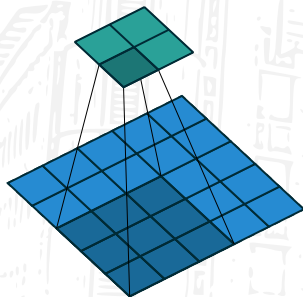
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

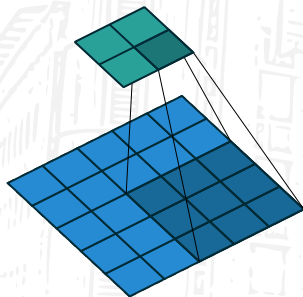
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

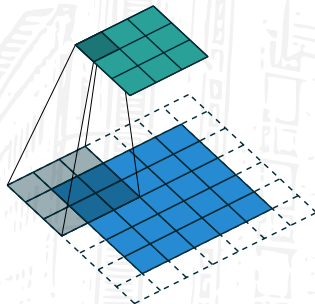
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1, o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

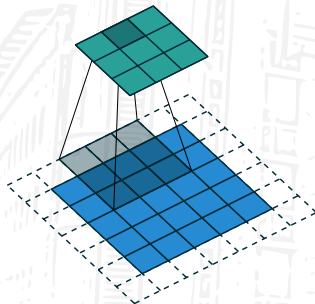
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1, o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$
- ▶ combinations, $o = \lfloor (i + 2p - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

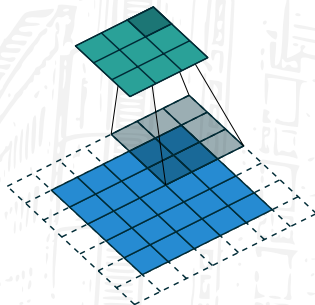
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1, o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$
- ▶ combinations, $o = \lfloor (i + 2p - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

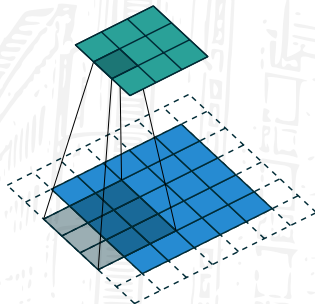
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$
- ▶ combinations, $o = \lfloor (i + 2p - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

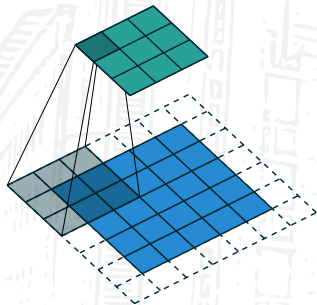
- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1, o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$
- ▶ combinations, $o = \lfloor (i + 2p - k)/s \rfloor + 1$

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

2D convolutions: refinements



Suppose

- ▶ input size: $i_1 \times i_2$,
- ▶ output size $o_1 \times o_2$,
- ▶ kernel size $k_1 \times k_2$.

For 2D convolutions we can consider **various extra operations**:

- ▶ full padding, $o_1 = i_1 + k_1 - 1$, $o_2 = i_2 + k_2 - 1$
- ▶ arbitrary padding, $o = i + 2p - k + 1$
- ▶ striding, $o = \lfloor (i - k)/s \rfloor + 1$
- ▶ combinations, $o = \lfloor (i + 2p - k)/s \rfloor + 1$

Next: image processing example (with padding).

Figures taken from Dumoulin and Visin [1], sources available at https://github.com/vdumoulin/conv_arithmetic

Example: recreated Lena and kernels

Recreated Lena (original)



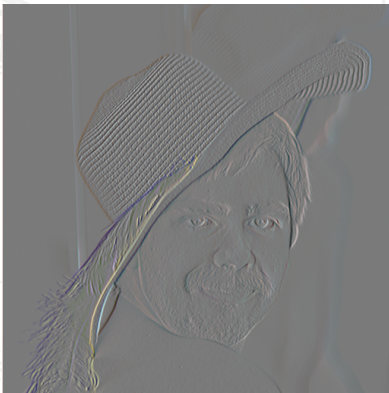
Identical transformation:

$$\mathbf{F} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Sobel vertical)



Sobel vertical:

$$\mathbf{F} = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Sobel horizontal)



Sobel horizontal:

$$\mathbf{F} = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Laplace edge detection)



Laplace:

$$\mathbf{F} = \frac{1}{6} \begin{pmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Gaussian blur)



Average (blur) $\times 1$:

$$\mathbf{F} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Gaussian blur & Laplace)



Average (blur) $\times 1$:

$$\mathbf{F} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Laplace:

$$\mathbf{F} = \frac{1}{6} \begin{pmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Gaussian blur twice)



Average (blur) $\times 2$:

$$\mathbf{F} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Gaussian blur twice & Laplace)



Average (blur) $\times 2$:

$$\mathbf{F} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Laplace:

$$\mathbf{F} = \frac{1}{6} \begin{pmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Gaussian blur thrice)



Average (blur) $\times 3$:

$$\mathbf{F} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / Gaussian blur thrice & Laplace)



Average (blur) $\times 3$:

$$\mathbf{F} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Laplace:

$$\mathbf{F} = \frac{1}{6} \begin{pmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{pmatrix}$$

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / channel average)



Channel average:

```
np.ones((3, 3, 3)) / 27
```

Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Example: recreated Lena and kernels

Recreated Lena (original / channel average, multiple blur & Laplace)



Morten Rieger Hannemose, Recreated Lena Picture, 2019, <https://mortenhannemose.github.io/lena/>

Channel average:

$$\text{np.ones}((3, 3, 3)) / 27$$

Average (blur) $\times 8$:

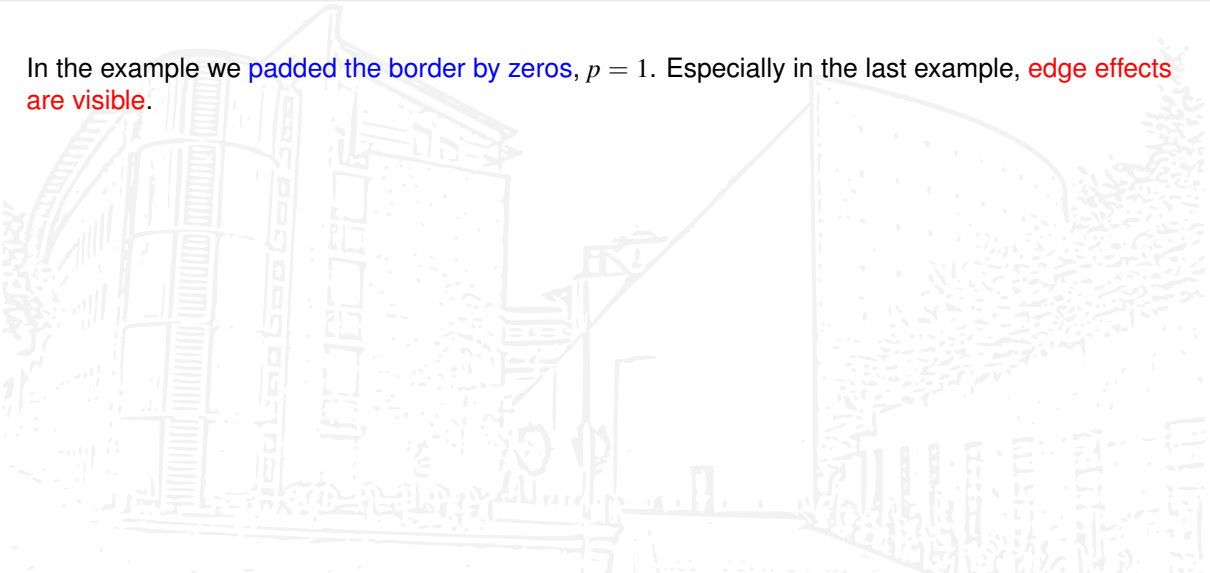
$$\mathbf{F} = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

Laplace:

$$\mathbf{F} = \frac{1}{6} \begin{pmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{pmatrix}$$

Treatment of border points

In the example we **padding the border by zeros**, $p = 1$. Especially in the last example, **edge effects** are visible.



Treatment of border points

In the example we **padding the border by zeros**, $p = 1$. Especially in the last example, **edge effects are visible**.

Other possible techniques include:

- ▶ **doubling/reflecting outer pixels** (with extra treatment for the corners)
- ▶ **identifying left and right & up and down** (the image lives on a torus, data is periodical)
- ▶ **identifying left upper with right lower pixels, similarly for up right and down left** (in 1D this gives a non-orientable manifold, the Möbius strip)
- ▶ **mix both last approaches** (results in a Klein bottle, another non-orientable manifold)

Treatment of border points

In the example we **padding the border by zeros**, $p = 1$. Especially in the last example, **edge effects are visible**.

Other possible techniques include:

- ▶ **doubling/reflecting outer pixels** (with extra treatment for the corners)
- ▶ **identifying left and right & up and down** (the image lives on a torus, data is periodical)
- ▶ **identifying left upper with right lower pixels, similarly for up right and down left** (in 1D this gives a non-orientable manifold, the Möbius strip)
- ▶ **mix both last approaches** (results in a Klein bottle, another non-orientable manifold)

These approaches result in periodic signals $\rightsquigarrow$ **Fourier analysis**

The Toeplitz connection

In **2D**: convolution / cross-correlation with **rotated kernel** $\mathbf{Y} = \mathbf{X} * \mathbf{F} = \mathbf{X} \star \text{np.rot90}(\mathbf{F}, 2)$, e.g.,

$$\begin{pmatrix} y_{1,1} & y_{1,2} \\ y_{2,1} & y_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} * \begin{pmatrix} f_{1,1} & f_{1,2} \\ f_{2,1} & f_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \star \begin{pmatrix} f_{2,2} & f_{2,1} \\ f_{1,2} & f_{1,1} \end{pmatrix}$$

The Toeplitz connection

In **2D**: convolution / cross-correlation with **rotated kernel** $\mathbf{Y} = \mathbf{X} * \mathbf{F} = \mathbf{X} \star \text{np.rot90}(\mathbf{F}, 2)$, e.g.,

$$\begin{pmatrix} y_{1,1} & y_{1,2} \\ y_{2,1} & y_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} * \begin{pmatrix} f_{1,1} & f_{1,2} \\ f_{2,1} & f_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \star \begin{pmatrix} f_{2,2} & f_{2,1} \\ f_{1,2} & f_{1,1} \end{pmatrix}$$

w/ a **block Toeplitz matrix with Toeplitz blocks**,

$$\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{2,1} \\ y_{2,2} \end{pmatrix} = \left(\begin{array}{ccc|ccc|ccc} f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} \end{array} \right) \cdot \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ \hline x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ \hline x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}.$$

The Toeplitz connection

In **2D**: convolution / cross-correlation with **rotated kernel** $\mathbf{Y} = \mathbf{X} * \mathbf{F} = \mathbf{X} \star \text{np.rot90}(\mathbf{F}, 2)$, e.g.,

$$\begin{pmatrix} y_{1,1} & y_{1,2} \\ y_{2,1} & y_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} * \begin{pmatrix} f_{1,1} & f_{1,2} \\ f_{2,1} & f_{2,2} \end{pmatrix} = \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \end{pmatrix} \star \begin{pmatrix} f_{2,2} & f_{2,1} \\ f_{1,2} & f_{1,1} \end{pmatrix}$$

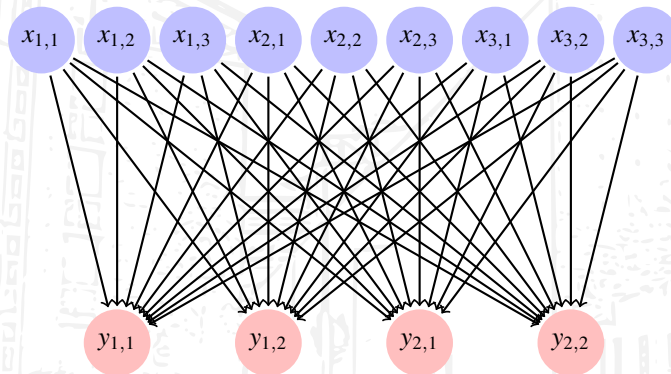
w/ a **block Toeplitz matrix with Toeplitz blocks**,

$$\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{2,1} \\ y_{2,2} \end{pmatrix} = \left(\begin{array}{ccc|ccc|ccc} f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & 0 & f_{2,2} & f_{2,1} & 0 & f_{1,2} & f_{1,1} \end{array} \right) \cdot \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ \hline x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ \hline x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}.$$

```
from scipy.signal import convolve2d
Y = convolve2d(X, F, mode='valid')
```

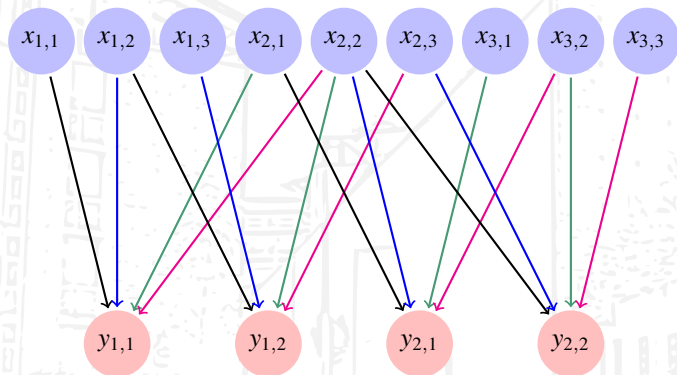
Weight sharing, pictorial understanding

Weight sharing: fully connected ($9 \times 4 = 36$)



Weight sharing, pictorial understanding

Weight sharing: fully connected ($9 \times 4 = 36$) vs. convolutional layer ($4 \times 4 = 16$ **shared**)



d D convolutions

Cross-correlation and convolution can be extended to d D for any $d \in \mathbb{N}$,

$$(\mathbf{X} \star \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 + i_1, \dots, n_d + i_d) \cdot \mathbf{F}(i_1, \dots, i_d),$$

$$(\mathbf{X} * \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 - i_1, \dots, n_d - i_d) \cdot \mathbf{F}(i_1, \dots, i_d).$$

d D convolutions

Cross-correlation and convolution can be extended to d D for any $d \in \mathbb{N}$,

$$(\mathbf{X} \star \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 + i_1, \dots, n_d + i_d) \cdot \mathbf{F}(i_1, \dots, i_d),$$

$$(\mathbf{X} * \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 - i_1, \dots, n_d - i_d) \cdot \mathbf{F}(i_1, \dots, i_d).$$

They are related by **flipping the kernel in every dimension**, since **multi-dimensional convolution is commutative**, $\mathbf{X} * \mathbf{F} = \mathbf{F} * \mathbf{X}$ (exercise).

d D convolutions

Cross-correlation and convolution can be extended to d D for any $d \in \mathbb{N}$,

$$(\mathbf{X} \star \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 + i_1, \dots, n_d + i_d) \cdot \mathbf{F}(i_1, \dots, i_d),$$

$$(\mathbf{X} * \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 - i_1, \dots, n_d - i_d) \cdot \mathbf{F}(i_1, \dots, i_d).$$

They are related by flipping the kernel in every dimension, since multi-dimensional convolution is commutative, $\mathbf{X} * \mathbf{F} = \mathbf{F} * \mathbf{X}$ (exercise).

- ▶ 3D convolution can be written using a block Toeplitz matrix with blocks that are block Toeplitz with blocks that are Toeplitz.
- ▶ d D convolution can be written using a block Toeplitz matrix with blocks that are block Toeplitz with blocks that are block Toeplitz ...

d D convolutions

Cross-correlation and convolution can be extended to d D for any $d \in \mathbb{N}$,

$$(\mathbf{X} \star \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 + i_1, \dots, n_d + i_d) \cdot \mathbf{F}(i_1, \dots, i_d),$$

$$(\mathbf{X} * \mathbf{F})(n_1, \dots, n_d) := \sum_{\mathbf{i}} \mathbf{X}(n_1 - i_1, \dots, n_d - i_d) \cdot \mathbf{F}(i_1, \dots, i_d).$$

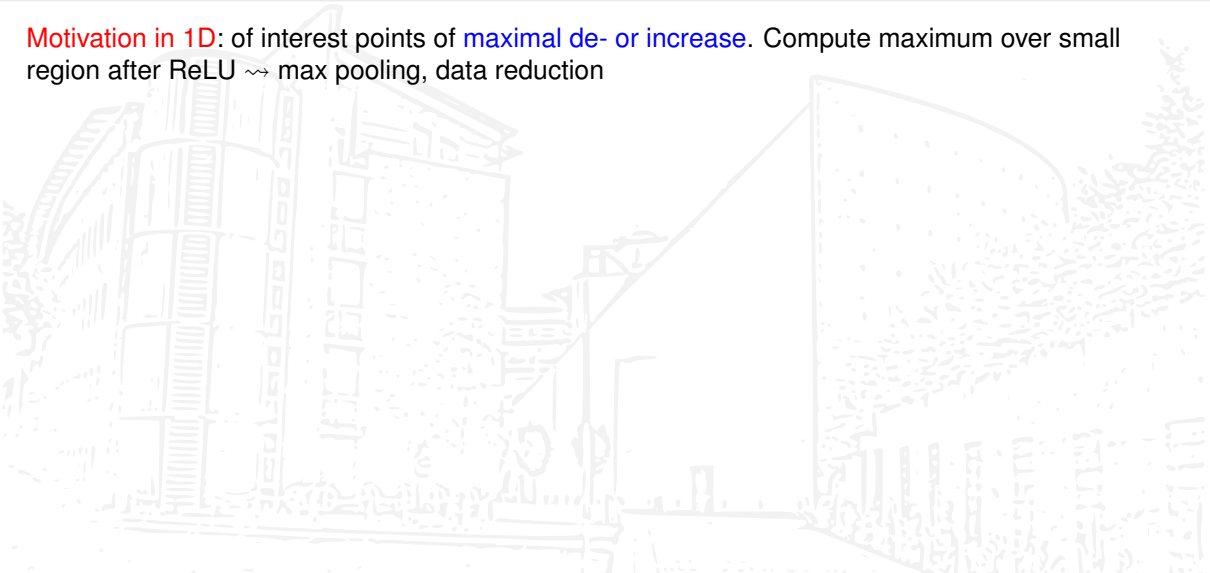
They are related by flipping the kernel in every dimension, since multi-dimensional convolution is commutative, $\mathbf{X} * \mathbf{F} = \mathbf{F} * \mathbf{X}$ (exercise).

- ▶ 3D convolution can be written using a block Toeplitz matrix with blocks that are block Toeplitz with blocks that are Toeplitz.
- ▶ d D convolution can be written using a block Toeplitz matrix with blocks that are block Toeplitz with blocks that are block Toeplitz ...

Many deep learning software packages include convolutions for dimension 1, 2, and 3. Engineers need code for 4D and more. Understanding convolutions may help to find a good job.

Pooling

Motivation in 1D: of interest points of **maximal de- or increase**. Compute maximum over small region after ReLU $\rightsquigarrow$ max pooling, data reduction



Pooling

Motivation in 1D: of interest points of **maximal de- or increase**. Compute maximum over small region after ReLU $\rightsquigarrow$ max pooling, data reduction

Generalizes to d D, we focus on 2D. Pooling is typically applied **after a convolutional layer**.

Pooling

Motivation in 1D: of interest points of **maximal de- or increase**. Compute maximum over small region after ReLU $\rightsquigarrow$ max pooling, data reduction

Generalizes to dD , we focus on 2D. Pooling is typically applied **after a convolutional layer**.

The most common pooling layers in 2D are given by

max pooling: mostly reduction of four neighbouring pixels to one that carries the maximal value.
For later convenience: **store “winning” indices** $\rightsquigarrow$ backpropagation

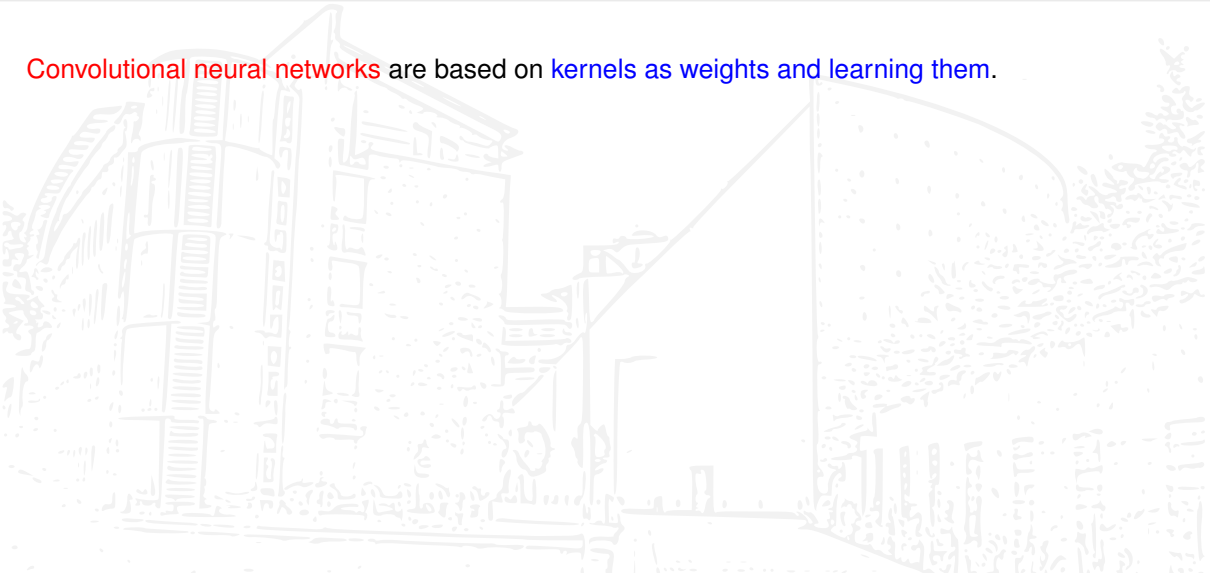
min pooling: basically like max pooling, only based on minimum. Again **storage of “winning” indices**

averaging: mostly reduction of four neighbouring pixels to one value that contains the **average of the four**

dropping: only every second, third, $\dots$ row and column is used, this is usually implemented as part of the convolution (**convolution with strides**) as there is no need to compute quantities that are dropped afterwards

Learning kernels

Convolutional neural networks are based on kernels as weights and learning them.



Learning kernels

Convolutional neural networks are based on **kernels as weights and learning them**.

Differences to **fully connected ANN**:

- ▶ Feedforward: simply use **convolution** in place of the structured weight matrix. ✓
(We'll learn faster ways to implement convolutions.)

Learning kernels

Convolutional neural networks are based on **kernels as weights and learning them**.

Differences to **fully connected ANN**:

- ▶ Feedforward: simply use **convolution** in place of the structured weight matrix. ✓
(We'll learn faster ways to implement convolutions.)
- ▶ **Backpropagation of the δ vectors**: we could use the transposed structured weight matrix, but maybe there are better ways?

Learning kernels

Convolutional neural networks are based on **kernels as weights and learning them**.

Differences to **fully connected ANN**:

- ▶ Feedforward: simply use **convolution** in place of the structured weight matrix. ✓
(We'll learn faster ways to implement convolutions.)
- ▶ **Backpropagation of the δ vectors**: we could use the transposed structured weight matrix, but maybe there are better ways?
- ▶ **Update of the kernels** vs. known update of the weight matrices: updating a sparse block Toeplitz structured matrix by a rank-one update would most probably not result in a new structured matrix of the same type. How do we proceed here?

Learning kernels

Convolutional neural networks are based on kernels as weights and learning them.

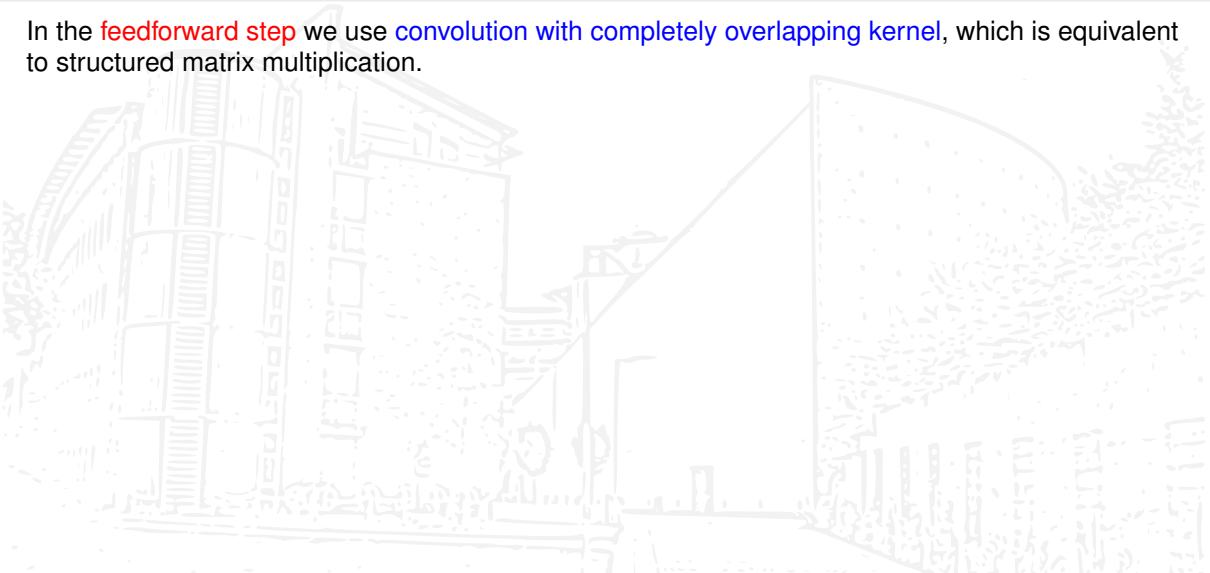
Differences to fully connected ANN:

- ▶ Feedforward: simply use convolution in place of the structured weight matrix. ✓
(We'll learn faster ways to implement convolutions.)
- ▶ Backpropagation of the δ vectors: we could use the transposed structured weight matrix, but maybe there are better ways?
- ▶ Update of the kernels vs. known update of the weight matrices: updating a sparse block Toeplitz structured matrix by a rank-one update would most probably not result in a new structured matrix of the same type. How do we proceed here?

In the following, we consider these three differences in this order.

Feedforward

In the **feedforward step** we use **convolution with completely overlapping kernel**, which is equivalent to structured matrix multiplication.



Feedforward

In the **feedforward step** we use **convolution with completely overlapping kernel**, which is equivalent to structured matrix multiplication. **In 1D** we use `y = np.convolve(x, f, mode='valid')`,

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_{n-k+1} \end{pmatrix} = \mathbf{x} * \mathbf{f} = \mathbf{T}\mathbf{x} := \begin{pmatrix} f_k & \dots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \dots & f_1 \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix},$$

Feedforward

In the **feedforward step** we use **convolution with completely overlapping kernel**, which is equivalent to structured matrix multiplication. **In 1D** we use `y = np.convolve(x, f, mode='valid')`,

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_{n-k+1} \end{pmatrix} = \mathbf{x} * \mathbf{f} = \mathbf{T}\mathbf{x} := \begin{pmatrix} f_k & \dots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \dots & f_1 \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix},$$

in 2D we use SciPy's `Y = convolve2d(X, F, mode='valid')`,

$$\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{2,1} \\ y_{2,2} \end{pmatrix} = \begin{pmatrix} f_{2,2} & f_{2,1} & 0 & | & f_{1,2} & f_{1,1} & 0 & | & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & | & 0 & f_{1,2} & f_{1,1} & | & 0 & 0 & 0 \\ 0 & 0 & 0 & | & f_{2,2} & f_{2,1} & 0 & | & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & | & 0 & f_{2,2} & f_{2,1} & | & 0 & f_{1,2} & f_{1,1} \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ \hline x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ \hline x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}.$$

Feedforward

In the **feedforward step** we use **convolution with completely overlapping kernel**, which is equivalent to structured matrix multiplication. **In 1D** we use `y = np.convolve(x, f, mode='valid')`,

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_{n-k+1} \end{pmatrix} = \mathbf{x} * \mathbf{f} = \mathbf{T}\mathbf{x} := \begin{pmatrix} f_k & \dots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \dots & f_1 \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix},$$

in 2D we use SciPy's `Y = convolve2d(X, F, mode='valid')`,

$$\begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{2,1} \\ y_{2,2} \end{pmatrix} = \begin{pmatrix} f_{2,2} & f_{2,1} & 0 & | & f_{1,2} & f_{1,1} & 0 & | & 0 & 0 & 0 \\ 0 & f_{2,2} & f_{2,1} & | & 0 & f_{1,2} & f_{1,1} & | & 0 & 0 & 0 \\ 0 & 0 & 0 & | & f_{2,2} & f_{2,1} & 0 & | & f_{1,2} & f_{1,1} & 0 \\ 0 & 0 & 0 & | & 0 & f_{2,2} & f_{2,1} & | & 0 & f_{1,2} & f_{1,1} \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ \hline x_{2,1} \\ x_{2,2} \\ x_{2,3} \\ \hline x_{3,1} \\ x_{3,2} \\ x_{3,3} \end{pmatrix}.$$

We define $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$ and $\mathbf{a} = a(\mathbf{z})$ as before.

Backpropagation

The multiplication with $\mathbf{W}^T$ in the **backpropagation of the δ vectors**, where $\mathbf{W}^T$ is given by

$$\begin{pmatrix} f_k & & & \\ \vdots & \ddots & & \\ f_1 & \dots & f_k & \\ & \ddots & \ddots & \ddots \\ & & f_1 & \dots & f_k \\ & & & \ddots & \vdots \\ & & & & f_1 \end{pmatrix}$$

or

$$\begin{pmatrix} f_{2,2} & 0 & 0 & 0 \\ f_{2,1} & f_{2,2} & 0 & 0 \\ 0 & f_{2,1} & 0 & 0 \\ f_{1,2} & 0 & f_{2,2} & 0 \\ f_{1,1} & f_{1,2} & f_{2,1} & f_{2,2} \\ 0 & f_{1,1} & 0 & f_{2,1} \\ 0 & 0 & f_{1,2} & 0 \\ 0 & 0 & f_{1,1} & f_{1,2} \\ 0 & 0 & 0 & f_{1,1} \end{pmatrix},$$

Backpropagation

The multiplication with $\mathbf{W}^T$ in the backpropagation of the δ vectors, where $\mathbf{W}^T$ is given by

$$\begin{pmatrix} f_k & & & \\ \vdots & \ddots & & \\ f_1 & \dots & f_k & \\ & \ddots & \ddots & \ddots \\ & & f_1 & \dots & f_k \\ & & & \ddots & \vdots \\ & & & & f_1 \end{pmatrix} \quad \text{or} \quad \begin{pmatrix} f_{2,2} & 0 & 0 & 0 \\ f_{2,1} & f_{2,2} & 0 & 0 \\ 0 & f_{2,1} & 0 & 0 \\ f_{1,2} & 0 & f_{2,2} & 0 \\ f_{1,1} & f_{1,2} & f_{2,1} & f_{2,2} \\ 0 & f_{1,1} & 0 & f_{2,1} \\ 0 & 0 & f_{1,2} & 0 \\ 0 & 0 & f_{1,1} & f_{1,2} \\ 0 & 0 & 0 & f_{1,1} \end{pmatrix},$$

can be interpreted as a **1D** and a **2D convolution**, respectively. This time it is the **full convolution** (i.e., with zero-padded input) and with **flipped kernel (1D)** or **doubly flipped = rotated kernel (2D)**.

Backpropagation

The multiplication with $\mathbf{W}^T$ in the backpropagation of the δ vectors, where $\mathbf{W}^T$ is given by

$$\begin{pmatrix} f_k & & & \\ \vdots & \ddots & & \\ f_1 & \dots & f_k & \\ & \ddots & \ddots & \ddots \\ & & f_1 & \dots & f_k \\ & & & \ddots & \vdots \\ & & & & f_1 \end{pmatrix} \quad \text{or} \quad \begin{pmatrix} f_{2,2} & 0 & 0 & 0 \\ f_{2,1} & f_{2,2} & 0 & 0 \\ 0 & f_{2,1} & 0 & 0 \\ f_{1,2} & 0 & f_{2,2} & 0 \\ f_{1,1} & f_{1,2} & f_{2,1} & f_{2,2} \\ 0 & f_{1,1} & 0 & f_{2,1} \\ 0 & 0 & f_{1,2} & 0 \\ 0 & 0 & f_{1,1} & f_{1,2} \\ 0 & 0 & 0 & f_{1,1} \end{pmatrix},$$

can be interpreted as a 1D and a 2D convolution, respectively. This time it is the full convolution (i.e., with zero-padded input) and with flipped kernel (1D) or doubly flipped = rotated kernel (2D).

In the d D case we have to pad and flip along every dimension.

Kernel update

To compute the **kernel update** we reconsider the derivation. Let $\delta = \partial C / \partial \mathbf{z}$ and $\mathbf{z} = \mathbf{a} * \mathbf{f} + \mathbf{b}$ be computed using a convolution with kernel $\mathbf{f}$.

$\rightsquigarrow$ corresponds to more general case: activated $\mathbf{a}$ instead of input $\mathbf{x}$

Kernel update

To compute the **kernel update** we reconsider the derivation. Let $\delta = \partial C / \partial \mathbf{z}$ and $\mathbf{z} = \mathbf{a} * \mathbf{f} + \mathbf{b}$ be computed using a convolution with kernel $\mathbf{f}$.

$\rightsquigarrow$ corresponds to more general case: activated $\mathbf{a}$ instead of input $\mathbf{x}$

We already know how to compute (the multi-dimensional array) δ using backpropagation. We write the convolution using **multi-indices** $\mathbf{i} = (i_1, \dots, i_d)$, $\mathbf{j} = (j_1, \dots, j_d)$, and $\mathbf{k} = (k_1, \dots, k_d)$,

$$z_{\mathbf{i}} = (\mathbf{a} * \mathbf{f})_{\mathbf{i}} + b_{\mathbf{i}} = \sum_{\mathbf{j}=\mathbf{e}}^{\mathbf{k}} a_{\mathbf{i}-\mathbf{j}} f_{\mathbf{j}} + b_{\mathbf{i}}, \quad \mathbf{f} \in \mathbb{R}^{\mathbf{k}}, \quad \mathbf{e} = (1, \dots, 1).$$

Kernel update

To compute the **kernel update** we reconsider the derivation. Let $\delta = \partial C / \partial \mathbf{z}$ and $\mathbf{z} = \mathbf{a} * \mathbf{f} + \mathbf{b}$ be computed using a convolution with kernel $\mathbf{f}$.

$\rightsquigarrow$ corresponds to more general case: activated $\mathbf{a}$ instead of input $\mathbf{x}$

We already know how to compute (the multi-dimensional array) δ using backpropagation. We write the convolution using **multi-indices** $\mathbf{i} = (i_1, \dots, i_d)$, $\mathbf{j} = (j_1, \dots, j_d)$, and $\mathbf{k} = (k_1, \dots, k_d)$,

$$z_{\mathbf{i}} = (\mathbf{a} * \mathbf{f})_{\mathbf{i}} + b_{\mathbf{i}} = \sum_{\mathbf{j}=\mathbf{e}}^{\mathbf{k}} a_{\mathbf{i}-\mathbf{j}} f_{\mathbf{j}} + b_{\mathbf{i}}, \quad \mathbf{f} \in \mathbb{R}^{\mathbf{k}}, \quad \mathbf{e} = (1, \dots, 1).$$

Let $\tilde{\delta}$ denote δ flipped in every dimension, $\tilde{\delta}_{\mathbf{k}-\mathbf{i}+\mathbf{e}} = \delta_{\mathbf{i}}$.

Kernel update

To compute the **kernel update** we reconsider the derivation. Let $\delta = \partial C / \partial \mathbf{z}$ and $\mathbf{z} = \mathbf{a} * \mathbf{f} + \mathbf{b}$ be computed using a convolution with kernel $\mathbf{f}$.

$\rightsquigarrow$ corresponds to more general case: activated $\mathbf{a}$ instead of input $\mathbf{x}$

We already know how to compute (the multi-dimensional array) δ using backpropagation. We write the convolution using **multi-indices** $\mathbf{i} = (i_1, \dots, i_d)$, $\mathbf{j} = (j_1, \dots, j_d)$, and $\mathbf{k} = (k_1, \dots, k_d)$,

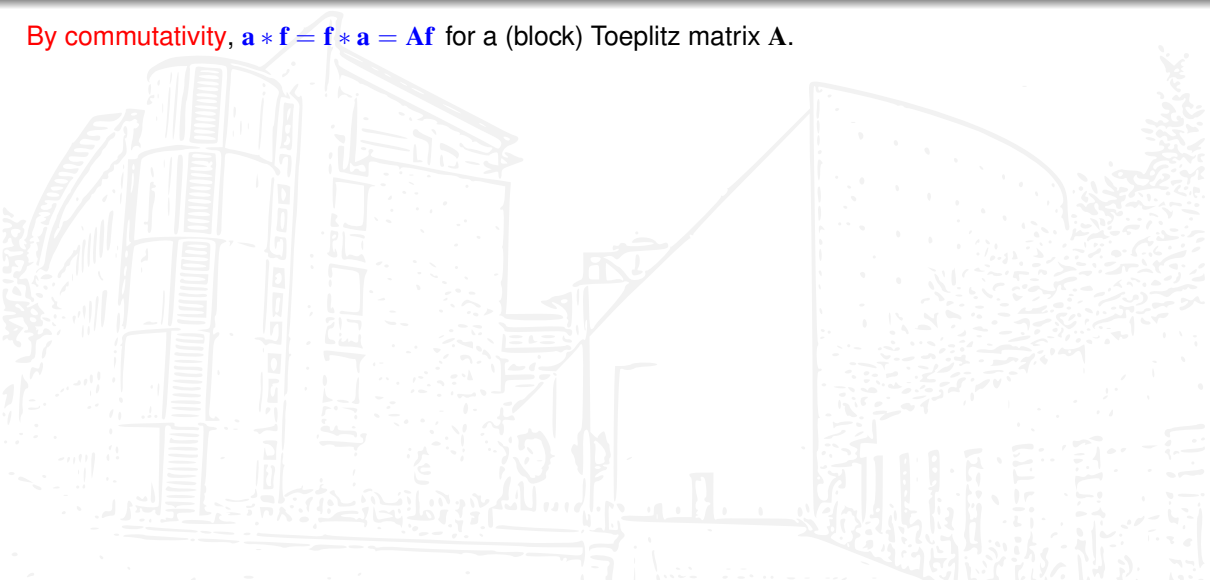
$$z_{\mathbf{i}} = (\mathbf{a} * \mathbf{f})_{\mathbf{i}} + b_{\mathbf{i}} = \sum_{\mathbf{j}=\mathbf{e}}^{\mathbf{k}} a_{\mathbf{i}-\mathbf{j}} f_{\mathbf{j}} + b_{\mathbf{i}}, \quad \mathbf{f} \in \mathbb{R}^{\mathbf{k}}, \quad \mathbf{e} = (1, \dots, 1).$$

Let $\tilde{\delta}$ denote δ **flipped in every dimension**, $\tilde{\delta}_{\mathbf{k}-\mathbf{i}+\mathbf{e}} = \delta_{\mathbf{i}}$. The **kernel update** is given by the **convolution** of the **flipped** $\mathbf{a}$ ($\tilde{a}_{\mathbf{j}-\mathbf{i}} := a_{\mathbf{i}-\mathbf{j}}$) with the **kernel** δ or the **flipped convolution** of $\mathbf{a}$ with the **flipped kernel** $\tilde{\delta}$,

$$\frac{\partial C}{\partial f_{\mathbf{j}}} = \sum_{\mathbf{i}} \frac{\partial z_{\mathbf{i}}}{\partial f_{\mathbf{j}}} \frac{\partial C}{\partial z_{\mathbf{i}}} = \sum_{\mathbf{i}} a_{\mathbf{i}-\mathbf{j}} \delta_{\mathbf{i}} (= (\tilde{\mathbf{a}} * \delta)_{\mathbf{j}}) = \sum_{\mathbf{i}} a_{\mathbf{i}-\mathbf{j}} \tilde{\delta}_{\mathbf{k}-\mathbf{i}+\mathbf{e}} = (\mathbf{a} * \tilde{\delta})_{\mathbf{k}-\mathbf{j}+\mathbf{e}}.$$

Kernel update using Toeplitz structure

By commutativity, $\mathbf{a} * \mathbf{f} = \mathbf{f} * \mathbf{a} = \mathbf{A}\mathbf{f}$ for a (block) Toeplitz matrix $\mathbf{A}$.



Kernel update using Toeplitz structure

By **commutativity**, $\mathbf{a} * \mathbf{f} = \mathbf{f} * \mathbf{a} = \mathbf{A}\mathbf{f}$ for a (block) Toeplitz matrix $\mathbf{A}$. “Our” derivative of $\mathbf{z}$ with respect to the kernel $\mathbf{f}$ (we use gradients in place of the derivative) is given by

$$\frac{\partial \mathbf{z}}{\partial \mathbf{f}} = \mathbf{A}^\top \Rightarrow \frac{\partial C}{\partial \mathbf{f}} = \frac{\partial \mathbf{z}}{\partial \mathbf{f}} \frac{\partial C}{\partial \mathbf{z}} = \mathbf{A}^\top \boldsymbol{\delta} = \tilde{\mathbf{a}} * \boldsymbol{\delta}.$$

Kernel update using Toeplitz structure

By **commutativity**, $\mathbf{a} * \mathbf{f} = \mathbf{f} * \mathbf{a} = \mathbf{A}\mathbf{f}$ for a (block) Toeplitz matrix $\mathbf{A}$. “Our” derivative of $\mathbf{z}$ with respect to the kernel $\mathbf{f}$ (we use gradients in place of the derivative) is given by

$$\frac{\partial \mathbf{z}}{\partial \mathbf{f}} = \mathbf{A}^\top \Rightarrow \frac{\partial C}{\partial \mathbf{f}} = \frac{\partial \mathbf{z}}{\partial \mathbf{f}} \frac{\partial C}{\partial \mathbf{z}} = \mathbf{A}^\top \boldsymbol{\delta} = \tilde{\mathbf{a}} * \boldsymbol{\delta}.$$

We show **in 1D** how this result **is obtained** (observe $n > k, n > n - k + 1$), ($\rightsquigarrow$ 2D: exercise)

$$\mathbf{W}\mathbf{a} = \begin{pmatrix} f_k & \dots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \dots & f_1 \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}$$

Kernel update using Toeplitz structure

By commutativity, $\mathbf{a} * \mathbf{f} = \mathbf{f} * \mathbf{a} = \mathbf{A}\mathbf{f}$ for a (block) Toeplitz matrix $\mathbf{A}$. “Our” derivative of $\mathbf{z}$ with respect to the kernel $\mathbf{f}$ (we use gradients in place of the derivative) is given by

$$\frac{\partial \mathbf{z}}{\partial \mathbf{f}} = \mathbf{A}^\top \Rightarrow \frac{\partial C}{\partial \mathbf{f}} = \frac{\partial \mathbf{z}}{\partial \mathbf{f}} \frac{\partial C}{\partial \mathbf{z}} = \mathbf{A}^\top \boldsymbol{\delta} = \tilde{\mathbf{a}} * \boldsymbol{\delta}.$$

We show in 1D how this result is obtained (observe $n > k, n > n - k + 1$), ($\rightsquigarrow$ 2D: exercise)

$$\mathbf{W}\mathbf{a} = \begin{pmatrix} f_k & \dots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \dots & f_1 \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} a_k & \dots & a_1 \\ \vdots & \ddots & \vdots \\ a_n & \dots & a_{n-k+1} \end{pmatrix} \begin{pmatrix} f_1 \\ \vdots \\ f_k \end{pmatrix} = \mathbf{A}\mathbf{f}, \quad \mathbf{A} \in \mathbb{R}^{(n-k+1) \times k},$$

Kernel update using Toeplitz structure

By **commutativity**, $\mathbf{a} * \mathbf{f} = \mathbf{f} * \mathbf{a} = \mathbf{A}\mathbf{f}$ for a (block) Toeplitz matrix $\mathbf{A}$. “Our” derivative of $\mathbf{z}$ with respect to the kernel $\mathbf{f}$ (we use gradients in place of the derivative) is given by

$$\frac{\partial \mathbf{z}}{\partial \mathbf{f}} = \mathbf{A}^\top \Rightarrow \frac{\partial C}{\partial \mathbf{f}} = \frac{\partial \mathbf{z}}{\partial \mathbf{f}} \frac{\partial C}{\partial \mathbf{z}} = \mathbf{A}^\top \boldsymbol{\delta} = \tilde{\mathbf{a}} * \boldsymbol{\delta}.$$

We show in 1D how this result is obtained (observe $n > k, n > n - k + 1$), ($\rightsquigarrow$ 2D: exercise)

$$\mathbf{W}\mathbf{a} = \begin{pmatrix} f_k & \cdots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \cdots & f_1 \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} a_k & \cdots & a_1 \\ \vdots & \ddots & \vdots \\ a_n & \cdots & a_{n-k+1} \end{pmatrix} \begin{pmatrix} f_1 \\ \vdots \\ f_k \end{pmatrix} = \mathbf{A}\mathbf{f}, \quad \mathbf{A} \in \mathbb{R}^{(n-k+1) \times k},$$

thus, “our” derivative is

$$\frac{\partial C}{\partial \mathbf{f}} = \mathbf{A}^\top \boldsymbol{\delta} = \begin{pmatrix} a_k & \cdots & a_n \\ \vdots & \ddots & \vdots \\ a_1 & \cdots & a_{n-k+1} \end{pmatrix} \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_{n-k+1} \end{pmatrix}$$

Kernel update using Toeplitz structure

By **commutativity**, $\mathbf{a} * \mathbf{f} = \mathbf{f} * \mathbf{a} = \mathbf{A}\mathbf{f}$ for a (block) Toeplitz matrix $\mathbf{A}$. “Our” derivative of $\mathbf{z}$ with respect to the kernel $\mathbf{f}$ (we use gradients in place of the derivative) is given by

$$\frac{\partial \mathbf{z}}{\partial \mathbf{f}} = \mathbf{A}^\top \Rightarrow \frac{\partial C}{\partial \mathbf{f}} = \frac{\partial \mathbf{z}}{\partial \mathbf{f}} \frac{\partial C}{\partial \mathbf{z}} = \mathbf{A}^\top \boldsymbol{\delta} = \tilde{\mathbf{a}} * \boldsymbol{\delta}.$$

We show in 1D how this result is obtained (observe $n > k$, $n > n - k + 1$), ($\rightsquigarrow$ 2D: exercise)

$$\mathbf{W}\mathbf{a} = \begin{pmatrix} f_k & \cdots & f_1 & & \\ & \ddots & \ddots & \ddots & \\ & & f_k & \cdots & f_1 \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} a_k & \cdots & a_1 & & \\ \vdots & \ddots & \vdots & & \\ a_n & \cdots & a_{n-k+1} & & \end{pmatrix} \begin{pmatrix} f_1 \\ \vdots \\ f_k \end{pmatrix} = \mathbf{A}\mathbf{f}, \quad \mathbf{A} \in \mathbb{R}^{(n-k+1) \times k},$$

thus, “our” derivative is indeed given by

$$\frac{\partial C}{\partial \mathbf{f}} = \mathbf{A}^\top \boldsymbol{\delta} = \begin{pmatrix} a_k & \cdots & a_n \\ \vdots & \ddots & \vdots \\ a_1 & \cdots & a_{n-k+1} \end{pmatrix} \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_{n-k+1} \end{pmatrix} = \begin{pmatrix} \delta_{n-k+1} & \cdots & \delta_1 & 0 & \cdots \\ & \ddots & \ddots & \ddots & \ddots \\ \cdots & 0 & \delta_{n-k+1} & \cdots & \delta_1 \end{pmatrix} \begin{pmatrix} a_n \\ \vdots \\ a_1 \end{pmatrix} = \tilde{\mathbf{a}} * \boldsymbol{\delta}.$$

In detail: 1D kernel update

The **other expression**

$$\frac{\partial C}{\partial \mathbf{f}} = \widetilde{(\mathbf{a} * \tilde{\delta})}$$

is also **easily obtained**.

In detail: 1D kernel update

The **other expression**

$$\frac{\partial C}{\partial \mathbf{f}} = \widetilde{(\mathbf{a} * \boldsymbol{\delta})}$$

is also **easily obtained**. We utilize

$$\mathbf{A}^T \boldsymbol{\delta} = \begin{pmatrix} a_k & \cdots & a_n \\ \vdots & \ddots & \vdots \\ a_1 & \cdots & a_{n-k+1} \end{pmatrix} \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_{n-k+1} \end{pmatrix}$$

In detail: 1D kernel update

The **other expression**

$$\frac{\partial C}{\partial \mathbf{f}} = \widetilde{(\mathbf{a} * \boldsymbol{\delta})}$$

is also **easily obtained**. We utilize

$$\mathbf{A}^T \boldsymbol{\delta} = \begin{pmatrix} a_k & \cdots & a_n \\ \vdots & \ddots & \vdots \\ a_1 & \cdots & a_{n-k+1} \end{pmatrix} \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_{n-k+1} \end{pmatrix} = \begin{pmatrix} \cdots & 0 & \delta_1 & \cdots & \delta_{n-k+1} \\ \ddots & \ddots & \ddots & \ddots & \ddots \\ \delta_1 & \cdots & \delta_{n-k+1} & 0 & \cdots \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}.$$

In detail: 1D kernel update

The **other expression**

$$\frac{\partial C}{\partial \mathbf{f}} = \widetilde{(\mathbf{a} * \boldsymbol{\delta})}$$

is also **easily obtained**. We utilize

$$\mathbf{A}^T \boldsymbol{\delta} = \begin{pmatrix} a_k & \cdots & a_n \\ \vdots & \ddots & \vdots \\ a_1 & \cdots & a_{n-k+1} \end{pmatrix} \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_{n-k+1} \end{pmatrix} = \begin{pmatrix} \cdots & 0 & \delta_1 & \cdots & \delta_{n-k+1} \\ \ddots & \ddots & \ddots & \ddots & \ddots \\ \delta_1 & \cdots & \delta_{n-k+1} & 0 & \cdots \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix}.$$

Flipping the rows (denoted by $\mathbf{F}$, not to be confused with the kernel) gives

$$\mathbf{F} \frac{\partial C}{\partial \mathbf{f}} = \mathbf{F} \cdot (\mathbf{A}^T \boldsymbol{\delta}) = \begin{pmatrix} \delta_1 & \cdots & \delta_{n-k+1} & 0 & \cdots \\ \ddots & \ddots & \ddots & \ddots & \ddots \\ \cdots & 0 & \delta_1 & \cdots & \delta_{n-k+1} \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_n \end{pmatrix} = \mathbf{a} * \widetilde{\boldsymbol{\delta}},$$

i.e., the **convolution of $\mathbf{a}$ with the flipped $\boldsymbol{\delta}$** .

Summary of changes for convolutions

To summarize changes in feedforward, backpropagation, & update with convolutions:

Feedforward: we use a completely overlapping convolution with a kernel F , e.g.,

1D: `y = np.convolve(a, f, mode='valid')`

2D: `Y = convolve2d(A, F, mode='valid')`

Backpropagation: we use a full convolution with the flipped kernel, e.g.,

1D: `np.convolve(delta, f[::-1], mode='full')`

2D: `convolve2d(Delta, F[::-1,::-1], mode='full')`

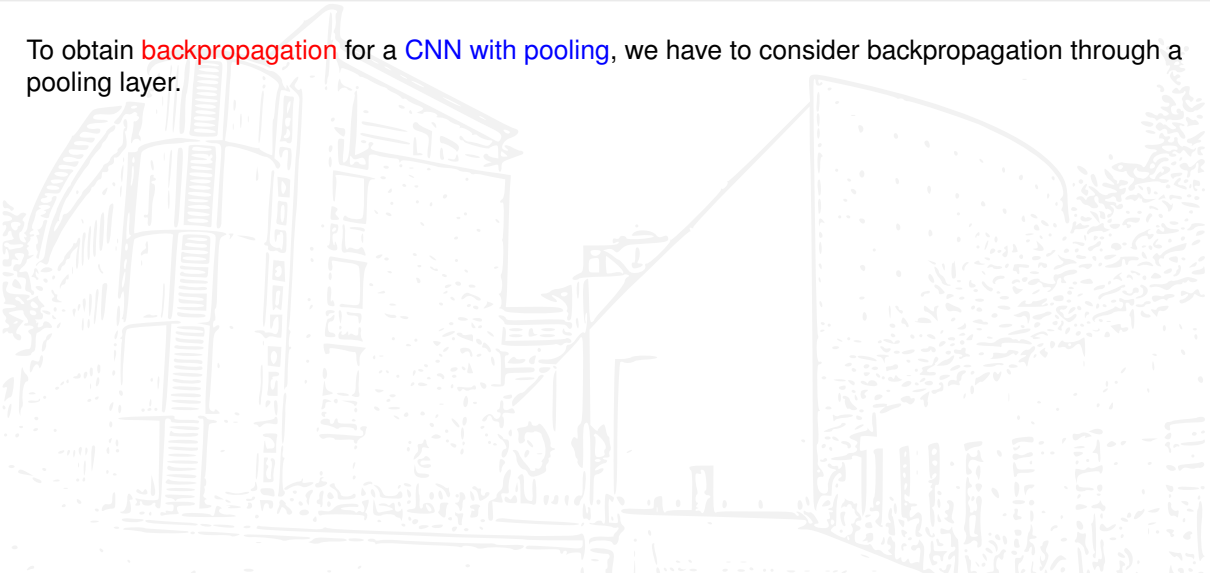
Kernel update: we use a completely overlapping convolution with either flipped activation or flipped δ vector (in which case we have to flip back), e.g.,

1D: `df = np.convolve(a[::-1], delta, mode='valid')`

2D: `dF = convolve2d(A[::-1,::-1], Delta, mode='valid')`

Pooling backpropagation

To obtain **backpropagation** for a **CNN with pooling**, we have to consider backpropagation through a pooling layer.



Pooling backpropagation

To obtain **backpropagation** for a **CNN with pooling**, we have to consider backpropagation through a pooling layer.

In **max pooling** we stored the values of the indices i where $a_i = \max_j a_j$. Suppose $i = 1$, then we can write

$$\max_j a_j = 1 \cdot a_1 + 0 \cdot a_2 + 0 \cdot a_3 + \dots$$

which has derivative 1 for a_1 and is zero elsewhere. The same holds true for **min pooling**.

Pooling backpropagation

To obtain **backpropagation** for a **CNN with pooling**, we have to consider backpropagation through a pooling layer.

In **max pooling** we stored the values of the indices i where $a_i = \max_j a_j$. Suppose $i = 1$, then we can write

$$\max_j a_j = 1 \cdot a_1 + 0 \cdot a_2 + 0 \cdot a_3 + \dots$$

which has derivative 1 for a_1 and is zero elsewhere. The same holds true for **min pooling**.

For **averaging** k numbers we can write

$$\text{average}_j(a_j) = \frac{1}{k} \sum_{j=1}^k a_j,$$

thus the derivative is $1/k$ for all j .

Comparison with a transposed FNN

In the exercises we change our implementation of FNN to treat **derivatives** in place of **gradients**, i.e., we **transpose** everything, e.g., use the same letters as in $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$, but now

$$\mathbf{z} = \mathbf{a}\mathbf{W} + \mathbf{b}, \quad \text{where} \quad \mathbf{a} \in \mathbb{R}^{1 \times n}, \quad \mathbf{W} \in \mathbb{R}^{n \times m}, \quad \mathbf{z}, \mathbf{b} \in \mathbb{R}^{1 \times m}, \quad \delta \in \mathbb{R}^{1 \times m}.$$

Comparison with a transposed FNN

In the exercises we change our implementation of FNN to treat **derivatives** in place of **gradients**, i.e., we **transpose** everything, e.g., use the same letters as in $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$, but now

$$\mathbf{z} = \mathbf{a}\mathbf{W} + \mathbf{b}, \quad \text{where } \mathbf{a} \in \mathbb{R}^{1 \times n}, \quad \mathbf{W} \in \mathbb{R}^{n \times m}, \quad \mathbf{z}, \mathbf{b} \in \mathbb{R}^{1 \times m}, \quad \delta \in \mathbb{R}^{1 \times m}.$$

Feedforward, **backpropagation**, & **update** for **dense** matrix-multiply and **convolution** are then:

task	dense	convolution
feedforward	$\mathbf{z} = \mathbf{a}\mathbf{W} + \mathbf{b}$	$\mathbf{z} = \mathbf{a} * \mathbf{f} + \mathbf{b}$ (valid)
backpropagation	$\delta \leftarrow \delta \mathbf{W}^T$	$\delta \leftarrow \delta * \tilde{\mathbf{f}}$ (full)
update $\mathbf{W}, \mathbf{f}$	$d\mathbf{W} = \mathbf{a}^T \delta$	$d\mathbf{f} = \tilde{\mathbf{a}} * \delta$ (valid)

Comparison with a transposed FNN

In the exercises we change our implementation of FNN to treat **derivatives** in place of **gradients**, i.e., we **transpose** everything, e.g., use the same letters as in $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$, but now

$$\mathbf{z} = \mathbf{a}\mathbf{W} + \mathbf{b}, \quad \text{where } \mathbf{a} \in \mathbb{R}^{1 \times n}, \quad \mathbf{W} \in \mathbb{R}^{n \times m}, \quad \mathbf{z}, \mathbf{b} \in \mathbb{R}^{1 \times m}, \quad \boldsymbol{\delta} \in \mathbb{R}^{1 \times m}.$$

Feedforward, **backpropagation**, & **update** for **dense** matrix-multiply and **convolution** are then:

task	dense	convolution
feedforward	$\mathbf{z} = \mathbf{a}\mathbf{W} + \mathbf{b}$	$\mathbf{z} = \mathbf{a} * \mathbf{f} + \mathbf{b}$ (valid)
backpropagation	$\boldsymbol{\delta} \leftarrow \boldsymbol{\delta}\mathbf{W}^\top$	$\boldsymbol{\delta} \leftarrow \boldsymbol{\delta} * \tilde{\mathbf{f}}$ (full)
update $\mathbf{W}, \mathbf{f}$	$d\mathbf{W} = \mathbf{a}^\top \boldsymbol{\delta}$	$d\mathbf{f} = \tilde{\mathbf{a}} * \boldsymbol{\delta}$ (valid)

CNN not only use convolutions, the **last layers** are **dense layers** for **classification** or **regression**.

A CNN for MNIST – load MNIST

```
1 import tensorflow as tf
2
3 print(tf.keras.backend.image_data_format())
4
5 from tensorflow.keras.datasets import mnist
6 from tensorflow.keras.utils import to_categorical
7
8 (x_train, y_train), (x_test, y_test) = mnist.load_data()
9 x_train = x_train[:, :, :, tf.newaxis] / 255.0
10 x_test = x_test[:, :, :, tf.newaxis] / 255.0
11 y_train, y_test = to_categorical(y_train), to_categorical(y_test)
```

A CNN for MNIST – define TF/Keras model

```
13 from tensorflow.keras.models import Sequential
14 from tensorflow.keras.layers import InputLayer, Flatten, Dense
15 from tensorflow.keras.layers import Conv2D, MaxPooling2D
16
17 model = Sequential()
18 model.add(InputLayer(shape=(28, 28, 1)))
19 model.add(Conv2D(32, (3, 3), activation='relu', kernel_initializer='he_uniform'))
20 model.add(MaxPooling2D((2, 2)))
21 model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
22 model.add(Conv2D(64, (3, 3), activation='relu', kernel_initializer='he_uniform'))
23 model.add(MaxPooling2D((2, 2)))
24 model.add(Flatten())
25 model.add(Dense(100, activation='relu', kernel_initializer='he_uniform'))
26 model.add(Dense(10))
```

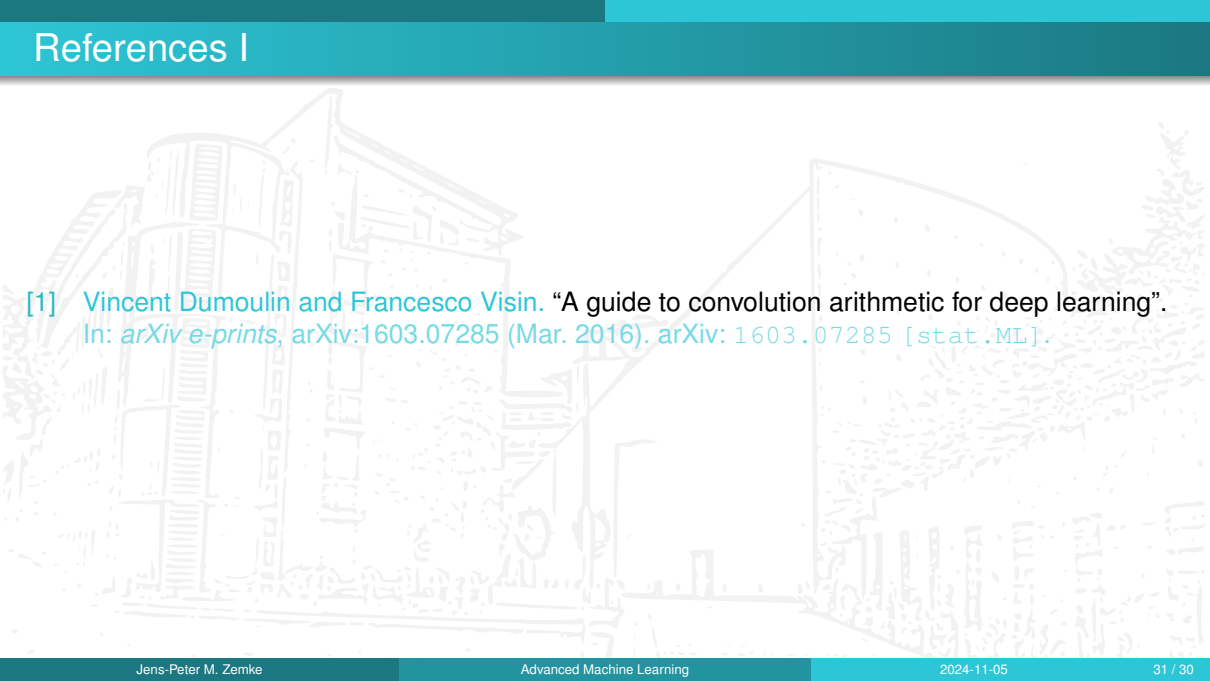
A CNN for MNIST – compile & train model

```
28 from tensorflow.keras.losses import CategoricalCrossentropy
29 from tensorflow.keras.optimizers import SGD
30
31 model.compile(
32     optimizer=SGD(learning_rate=0.01, momentum=0.9),
33     loss=CategoricalCrossentropy(from_logits=True), metrics=['accuracy'])
34 model.fit(x_train, y_train, epochs=10, batch_size=32, verbose=1)
```

A CNN for MNIST – evaluate model

```
36 import numpy as np
37
38 y = model.predict(x_test)
39 guess, label = np.argmax(y, axis=1), np.argmax(y_test, axis=1)
40 matched = guess == label
41 CNN_test_accuracy = np.sum(matched) * 100.0 / 10000.0
42 print('Accuracy_on_test_data:', CNN_test_accuracy)
43
44 import matplotlib.pyplot as plt
45
46 indices = np.arange(10000).astype(int)
47 for index in indices[np.logical_not(matched)]:
48     plt.imshow(x_test[index, :], 'gray')
49     plt.title(f'guess: {guess[index]}, label: {label[index]}')
50     plt.show()
```


References I

- 
- [1] Vincent Dumoulin and Francesco Visin. “A guide to convolution arithmetic for deep learning”. In: *arXiv e-prints*, arXiv:1603.07285 (Mar. 2016). arXiv: 1603.07285 [stat.ML].